Module Patterns: Layer Pattern

- **Context:**
  - All complex systems experience the need to develop and evolve portions of the system independently.
  - For this reason the developers of the system need a clear and well-documented separation of concerns, so that modules of the system may be independently developed and maintained.

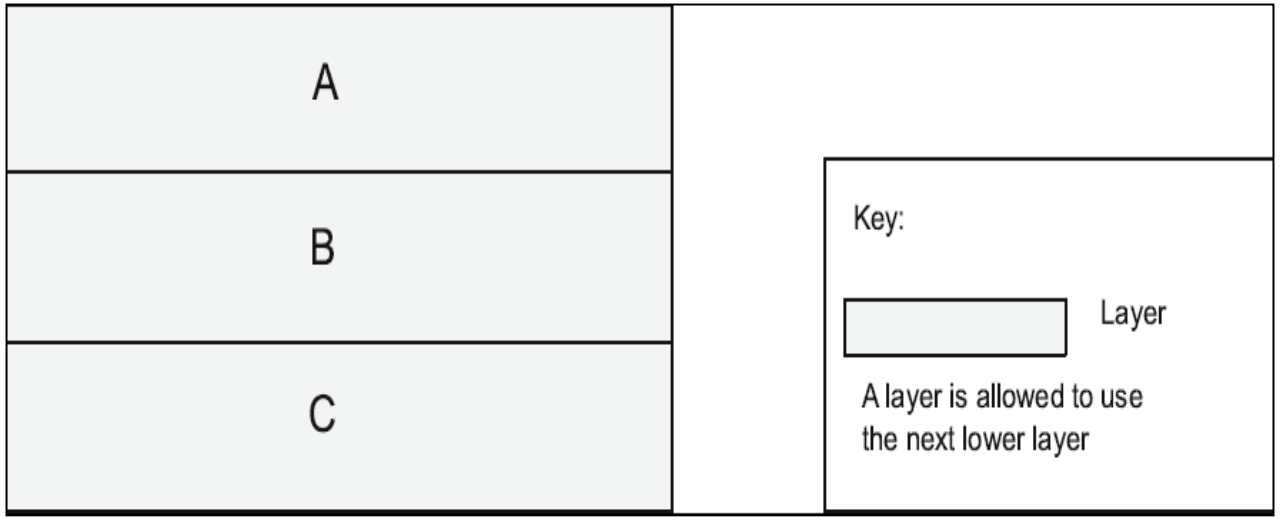
- **Problem:** The software needs to be segmented in such a way that the modules can be developed and evolved separately with little interaction among the parts, supporting portability, modifiability, and reuse.

- **Solution:**
  - To achieve this separation of concerns, the layered pattern divides the software into units called layers.
  - Each layer is a grouping of modules that offers a cohesive set of services. The usage must be unidirectional.
  - Layers completely partition a set of software, and each partition is exposed through a public interface.

Layer Pattern Example

Layers are almost always drawn as a stack of boxes.

The *allowed-to-use* relation is denoted by geometric adjacency and is read from the top down



Bridging: In some cases, modules in one layer might be required to directly use modules in a nonadjacent lower layer

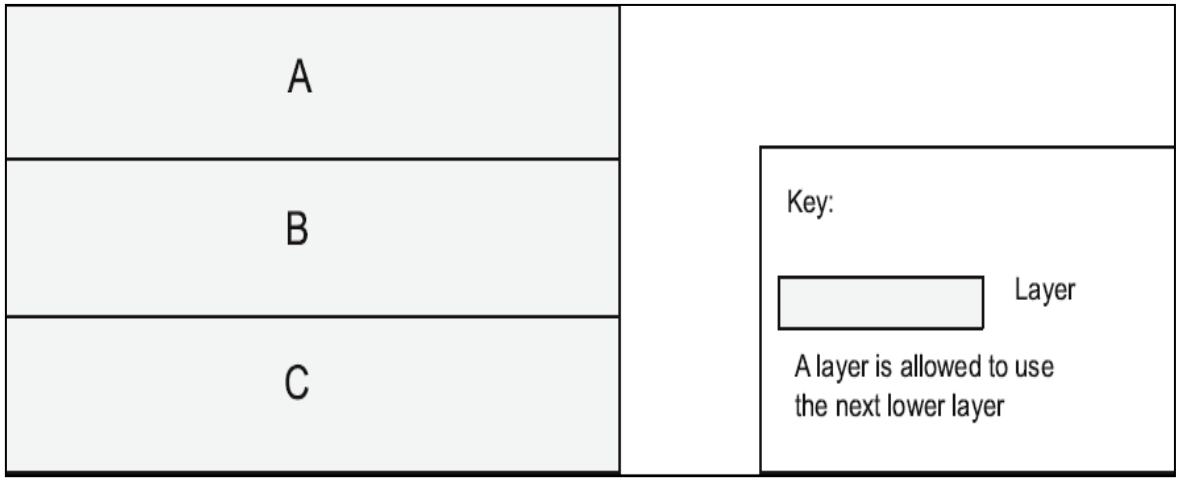
Upward usages are not allowed in this pattern.

Layer Pattern Solution

- Overview: The layered pattern defines layers (groupings of modules that offer a cohesive set of services) and a unidirectional *allowed-to-use* relation among the layers.
- Elements: *Layer*, a kind of module. The description of a layer should define what modules the layer contains.
- Relations: *Allowed to use*. The design should define what the layer usage rules are and any allowable exceptions.
- Constraints:
  - Every piece of software is allocated to exactly one layer.
  - There are at least two layers (but usually there are three or more).
  - The *allowed-to-use* relations should not be circular (i.e., a lower layer cannot use a layer above).
- Weaknesses:
  - The addition of layers adds up-front cost and complexity to a system.
  - Layers contribute a performance penalty.

Some Finer Points of Layers

- A layered architecture is one of the few places where connections among components can be shown by adjacency, and where "above" and "below" matter.
  - If you turn the diagram upside-down so that C is on top, this would represent a completely different design.
  - Diagrams that use arrows among the boxes to denote relations retain their semantic meaning no matter the orientation.



Some Finer Points of Layers

- The layered pattern is one of the most commonly used patterns in all of software engineering. However, some architects get it wrong.
  - First, sometimes it is impossible to look at a stack of boxes and tell whether layer bridging is allowed or not. That is, can a layer use any lower layer, or just the next lower one?
    - To resolve this; an architect has to include the answer in the key to the diagram's notation (something we recommend for all diagrams), as in Figure 13.2

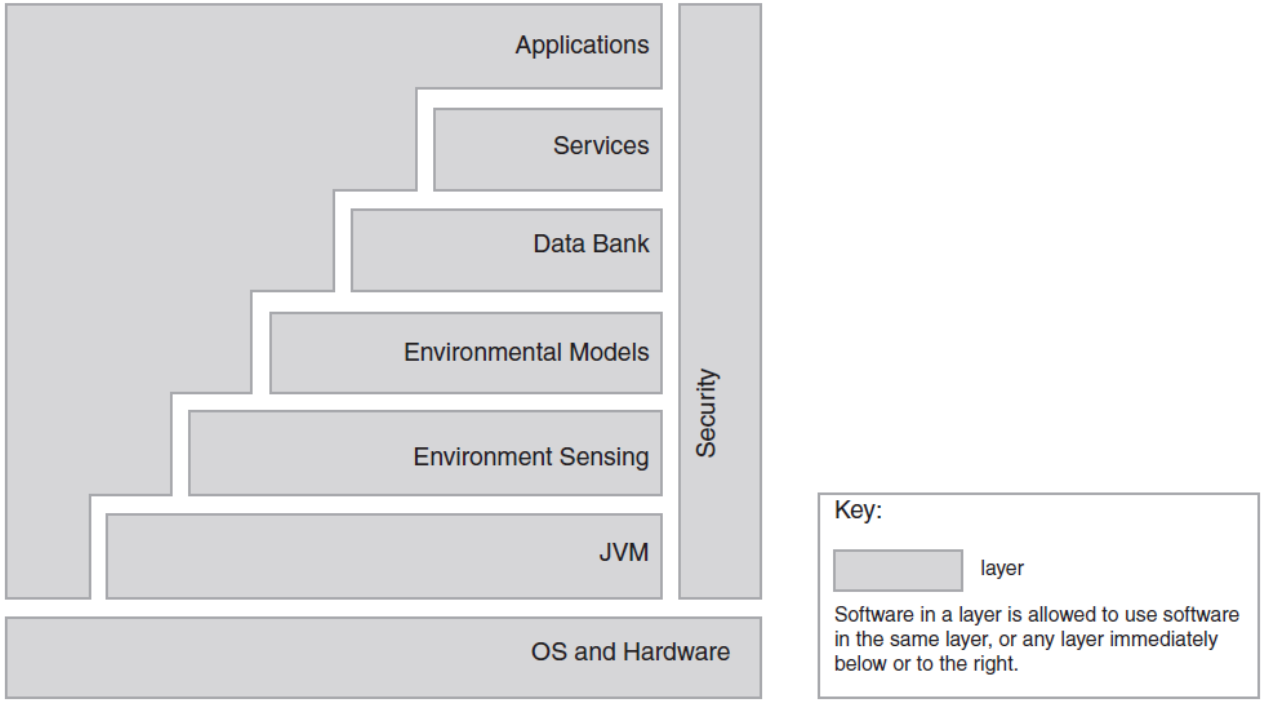


FIGURE 13.2 A simple layer diagram, with a simple key answering the uses question

Some Finer Points of Layers

- The layered pattern is one of the most commonly used patterns in all of software engineering. However, some architects get it wrong.
  - Second, any old set of boxes stacked on top of each other does not constitute a layered architecture.
  - For instance, look at the design shown in Figure 13.3, which uses arrows instead of adjacency to indicate the relationships among the boxes. Here, everything is allowed to use everything. This is decidedly **not** a **layered architecture**
  - The reason is that if Layer A is replaced by a different version, Layer C (which uses A) might well have to change. We **don't want** our **virtual machine layer** to **change every time** our **application layer changes**.

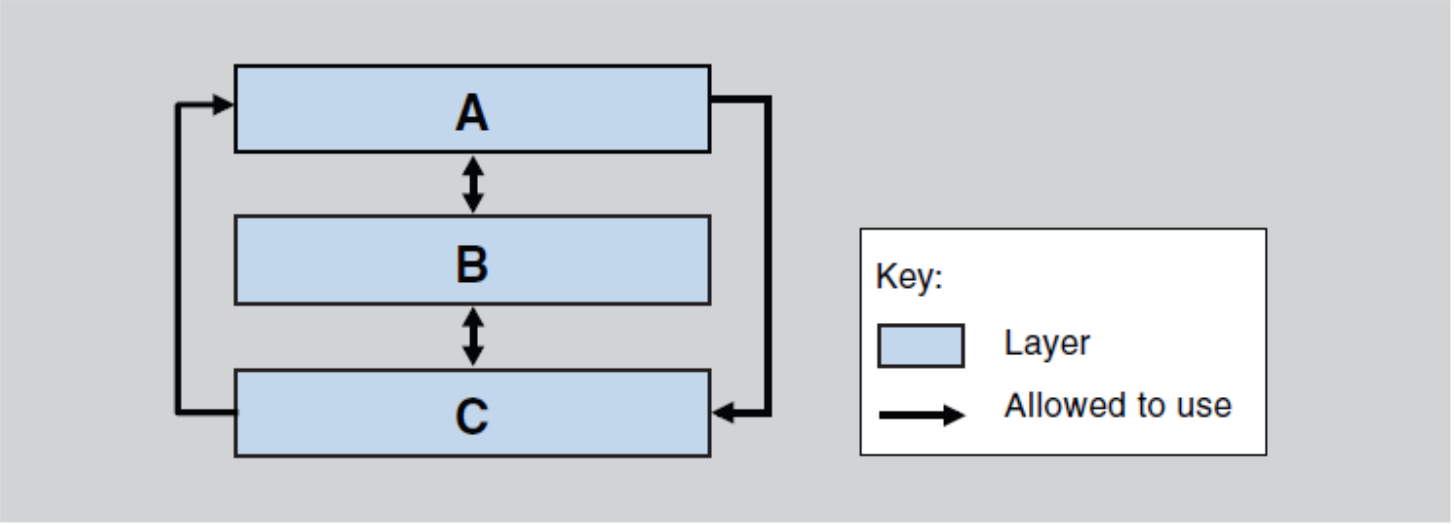
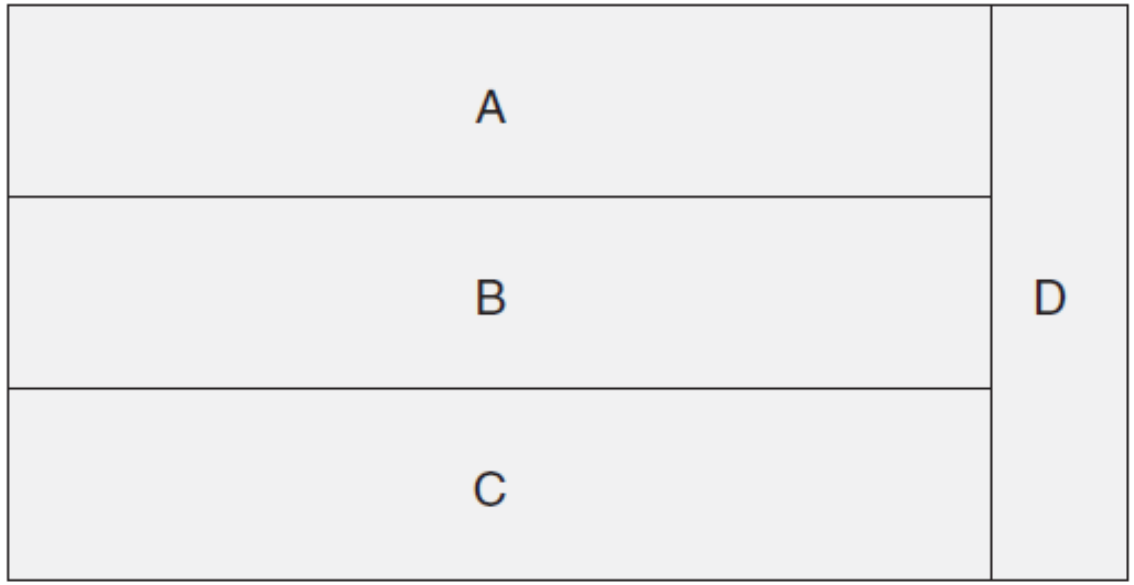


FIGURE 13.3 A wolf in layer's clothing

Some Finer Points of Layers

- What about this example? **Can you tell which layer uses which layer?**
  - For example: Does B use D?



Some Finer Points of Layers

- Sometimes layers are divided into segments denoting a finer-grained decomposition of the modules. This may occurs when a preexisting set of units, such as imported modules, share the same allowed-to-use relation.
  - When this happens, you have to specify what usage rules are in effect among the segments. Many usage rules are possible, but they must be made explicit.
  - In Figure 13.5, the top and the bottom layers are segmented. Segments of the top layer are not allowed to use each other, but segments of the bottom layer are.
  - If you draw the same diagram without the arrows, it will be harder to differentiate the different usage rules within segmented layers.

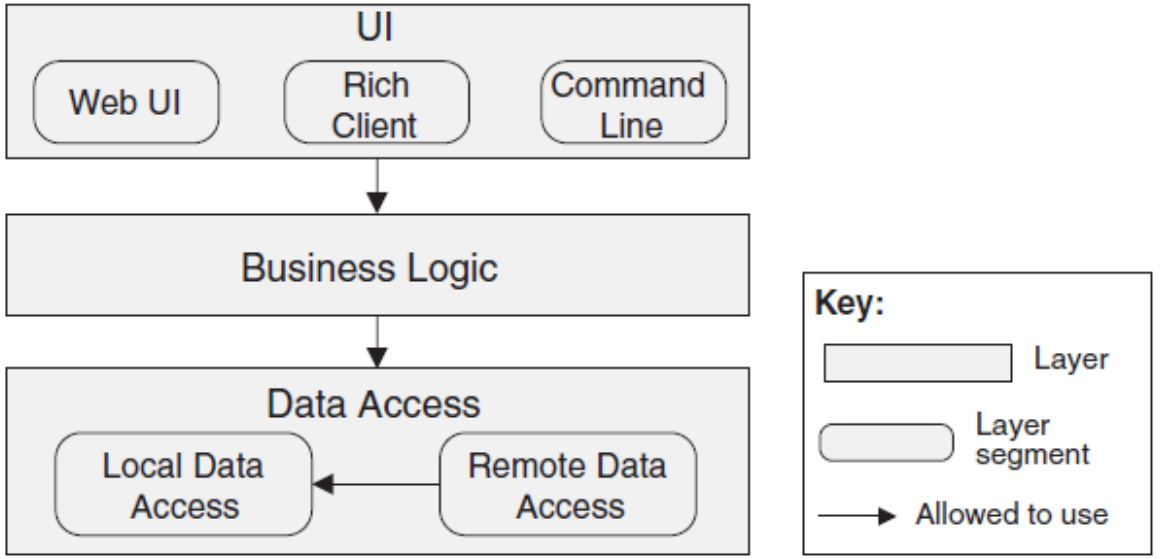


FIGURE 13.5 Layered design with segmented layers

Some Finer Points of Layers

- The most important point about layering is that a layer isn't allowed to use any layer above it.
- A module "uses" another module when it depends on the answer it gets back.
  - But a layer is allowed to make upward calls, if it isn't expecting an answer from them..